

Chapter 03-B: Design Patterns Catalog, Part 2

Core Learning Objective: Complete the pattern vocabulary you need in a low-level design interview: Singleton, Prototype, Decorator, Facade, Composite, Bridge, Flyweight, Template Method, Chain of Responsibility, Iterator, Mediator, Memento and Visitor. Every example below runs as written, ends with assertions, and says when Python makes the pattern unnecessary.

Part 1 (Chapter 03) covered Factory, Builder, Adapter, Proxy, Strategy, Observer, Command and State. Together the two chapters cover the patterns that actually appear in interviews and in real codebases; the remaining GoF patterns (Abstract Factory variants, Interpreter) are rarely asked and are covered by their names in the glossary at the end.

1. How to Use a Pattern Without Over-Engineering

A pattern is a named solution to a recurring force. Name the force first:

Force you feel	Pattern family	Pattern
"I need exactly one of these"	Creational	Singleton (usually: just pass one instance in)
"Creating it is expensive; I want a copy with changes"	Creational	Prototype
"I want to add behaviour without subclass explosion"	Structural	Decorator
"Callers should not know the five subsystems behind this"	Structural	Facade
"Part and whole should be treated the same"	Structural	Composite
"Two dimensions vary independently"	Structural	Bridge
"Millions of tiny objects share most of their data"	Structural	Flyweight
"The steps are fixed, the details vary"	Behavioural	Template Method
"Several handlers might deal with it; sender should not know which"	Behavioural	Chain of Responsibility
"Walk a collection without exposing its structure"	Behavioural	Iterator
"Objects talk to each other in a tangle"	Behavioural	Mediator
"I need undo or a snapshot"	Behavioural	Memento
"New operations over a stable set of node types"	Behavioural	Visitor

2. Creational Patterns

Singleton: one instance, safely

Python note. A module is imported once, so a module-level object (`settings = Settings()`) is already a singleton with no ceremony. The real objection to the pattern is not construction but **hidden global state**: tests cannot isolate it. Prefer creating one instance at start-up and passing it to whoever needs it.

Prototype: clone and tweak



Pitfall. `copy.copy` and `dataclasses.replace` copy references to mutable fields. Use `deepcopy` when the clone will mutate nested data, and remember it is slow for large graphs.

3. Structural Patterns

Decorator: add behaviour by wrapping



Four add-ons as subclasses would need up to 16 combinations; decorators need four classes. Do not confuse this with Python's `@decorator` syntax, which is a related idea applied to functions.

Facade: one simple door to a messy subsystem



A facade does not hide the subsystems; power users can still reach them. It narrows the common path.

Composite: treat a tree and a leaf the same



This is the pattern behind file systems, UI widget trees, menus and org charts. The In-Memory File System case study uses it.

Bridge: two dimensions that vary independently



With 3 shapes and 3 renderers, plain inheritance needs 9 classes; a bridge needs 6 and any new renderer works with every shape. Bridge is Strategy applied to a whole hierarchy.

Flyweight: share the immutable part



The shared (intrinsic) state lives in the flyweight; the per-use (extrinsic) state, such as position, stays outside it. Python's `sys.intern` and small-integer caching are built-in flyweights.

4. Behavioural Patterns

Template Method: fixed skeleton, variable steps

Template Method uses inheritance; if the steps are small you can pass them as functions (Strategy) and avoid the subclass.

Chain of Responsibility: pass it along until someone handles it

Web middleware (authentication, logging, rate limiting) is a chain of responsibility, with the twist that every link may also act before and after the next one.

Iterator: walk without exposing structure

Python bakes the pattern into the language: any object with `__iter__` works in `for`, comprehensions and `list()`. Write a generator unless you need the iterator to carry extra methods.

Mediator: replace many-to-many with many-to-one

Users know only the room, never each other, so n users need n links instead of n^2 . The risk is a mediator that grows into a god object: keep it to routing.

Memento: snapshot and restore

Store snapshots of immutable data (strings, tuples) so they cannot be corrupted later. For large state, store diffs (the Command pattern with an `undo()` method) rather than full copies.

Visitor: add operations without touching the node classes

The classic GoF Visitor uses `accept()` and double dispatch; `functools singledispatch` gives the same open set of operations in a few lines. Visitor pays off when the node types are stable and operations keep growing (compilers, linters); it hurts when new node types appear often, because every visitor must change.

5. Look-Alikes: Choosing Between Them

Pair	Real difference
Decorator vs Proxy	Decorator adds behaviour and is usually stacked by the client; a proxy controls access (lazy load, cache, auth) to one real object
Decorator vs Adapter	Adapter changes the interface; decorator keeps it
Facade vs Adapter	Facade simplifies many interfaces into a new one; adapter makes one interface match an expected one
Strategy vs State	Strategy is chosen from outside; State changes itself as the object moves between states
Strategy vs Template Method	Strategy composes (a swapped object); Template Method inherits (a subclass overrides steps)
Observer vs Mediator	Observer broadcasts to unknown listeners; Mediator centralises two-way coordination between known colleagues
Composite vs Decorator	Both nest, but composite aggregates many children, decorator wraps exactly one

6. Patterns the Language Already Gives You

Pattern	Python replacement
Singleton	A module-level instance
Iterator	Generators and <code>__iter__</code>
Strategy	Pass a function
Command	Pass a callable or <code>functools.partial</code>
Factory	Classes are callable; pass the class itself
Visitor	<code>functools.singledispatch</code>
Decorator (function form)	The <code>@</code> syntax and <code>functools.wraps</code>

Saying this in an interview shows judgment: use the pattern for the shared vocabulary, implement it with the lightest idiom.

7. Glossary of the Remaining Patterns

- **Abstract Factory:** a factory of factories producing families of related objects (for example one UI toolkit's button, menu and window together).
- **Interpreter:** represents a small language's grammar as classes and evaluates sentences; in practice you reach for a parser library.
- **Observer, Strategy, State, Command, Builder, Adapter, Proxy, Factory Method:** see Chapter 03.

Further Reading

- [Refactoring Guru: Composite](#)
- [Refactoring Guru: Visitor](#)
- [Python docs: functools.singledispatch](#)
- [Python docs: copy module](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Why is ``dataclasses.replace`` not a deep clone, and when does that bite?

It copies field references, so mutable fields (lists, dicts) are shared between the original and the copy; mutating one changes both. Use `copy.deepcopy` when the clone will modify nested data.

▼ 2. Decorator or subclassing for toppings on a drink?

Decorators: toppings combine freely at runtime, so n toppings need n classes rather than up to 2^n to the power n subclasses.

▼ 3. When does Visitor become a bad fit?

When new node types are added often: every existing visitor must gain a case for each new node. It suits stable node sets with growing operations.

▼ 4. Which of these does Python make mostly unnecessary, and why?

Singleton (a module-level instance is one object), Iterator (generators), Strategy and Command (functions are first-class), Visitor (`singledispatch`). The ideas remain useful as shared vocabulary even when the heavy class structure is not.